

DOCUMENTO TÉCNICA - SISTEMA DE PEDIDOS SAIMAZOOM

1. INTRODUCCIÓN

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema de pedidos Saimazoom, una plataforma automatizada para gestionar pedidos en un almacén. Permite a los clientes registrarse, autenticarse, realizar pedidos, consultarlos y cancelarlos, mientras robots buscan productos y repartidores los entregan. El objetivo es proporcionar una guía clara para el desarrollo, implementación y mantenimiento del sistema.

1.2 Alcance

El sistema Saimazoom incluye:

- Registro y autenticación de clientes.
- Creación, consulta y cancelación de pedidos.
- Búsqueda automatizada de productos por robots.
- Entrega de pedidos por repartidores.
- Comunicación asíncrona vía RabbitMQ.
- Registro de eventos en logs.

1.3 Definiciones, Acrónimos y Abreviaciones

- RabbitMQ: Sistema de mensajería para comunicación entre componentes.
- RPC: Remote Procedure Call, mecanismo cliente-servidor.
- Cliente: Usuario que realiza pedidos.
- Robot: Componente que busca productos en el almacén.
- Repartidor: Componente que entrega pedidos.
- Controlador: Componente central que coordina operaciones.

1.4 Referencias

- Documentación de RabbitMQ: <https://www.rabbitmq.com/>
- Python Pika: <https://pika.readthedocs.io/>
- Código fuente: Proporcionado en el contexto.

1.5 Audiencia

- Desarrolladores.
- Equipo de operaciones (configuración de RabbitMQ).
- Equipo de mantenimiento (auditoría y soporte).
- Stakeholders interesados en las funcionalidades.

2. DESCRIPCIÓN GENERAL

2.1 Perspectiva del Producto

Saimazoom es un sistema distribuido que automatiza la gestión de pedidos en un almacén, utilizando RabbitMQ para coordinar clientes, controlador, robots y repartidores. Es escalable y permite múltiples usuarios y componentes simultáneos.

2.2 Funciones del Producto

- Gestión de usuarios: Registro y autenticación.
- Gestión de pedidos: Creación, consulta y cancelación.
- Procesamiento de pedidos: Búsqueda de productos y entrega.
- Auditoría: Registro de eventos en logs.

2.3 Restricciones

- Dependencia de RabbitMQ.
- Colas no persistentes (durable=False, auto_delete=True).
- Interfaz de consola.
- Probabilidades simuladas para búsqueda (P_ALMACEN) y entrega (P_ENTREGA).
- Máximo tres intentos de entrega.

2.4 Suposiciones y Dependencias Suposiciones:

- Los clientes usan una terminal.
- Los productos solicitados son válidos (simulados). Dependencias:
- Python 3.6+ con pika, uuid, json, logging.
- Servidor RabbitMQ configurado.
- Archivo config.py con parámetros.

3. REQUISITOS ESPECÍFICOS

3.1 Requisitos Funcionales

1. Controlador

- **RF1.1:** El controlador debe gestionar los pedidos y coordinar la comunicación con los clientes, robots y repartidores a través de colas de mensajes en RabbitMQ.
- **RF1.2:** El controlador debe mantener un registro de todos los pedidos realizados, incluyendo su estado actual.
- **RF1.3:** El controlador debe mantener un registro de los clientes y asociar los pedidos a cada cliente.
- **RF1.4:** El controlador debe rechazar pedidos de clientes no registrados con un mensaje indicando que no están registrados.
- **RF1.5:** El controlador debe persistir su estado (pedidos y clientes) en todo momento, ya sea en ficheros (usando Pickle o CSV) o en una base de datos, y guardar los datos antes de finalizar la ejecución.
- **RF1.6:** El controlador debe recibir confirmaciones o errores de los robots y repartidores para actualizar el estado de los pedidos.

2. Clientes

- **RF2.1:** Los clientes deben poder registrarse en el sistema enviando un mensaje al controlador.
- **RF2.2:** Los clientes registrados deben poder crear pedidos, consultar su estado y cancelarlos (si no han sido asignados a un repartidor).
- **RF2.3:** El sistema debe proporcionar una interfaz de línea de comandos (CLI) para simular el comportamiento de un cliente desde la terminal.

3. Robots

- **RF3.1:** Los robots deben escuchar peticiones del controlador desde una cola de mensajes específica.
- **RF3.2:** Los robots deben simular la búsqueda de productos en el almacén con una espera aleatoria de 5 a 10 segundos.
- **RF3.3:** Los robots deben simular el éxito o fallo de la búsqueda de un producto con una probabilidad configurable (p_almacen).

- **RF3.4:** En caso de éxito, los robots deben notificar al controlador que el producto ha sido movido a la cinta transportadora.
- **RF3.5:** En caso de fallo, los robots deben notificar al controlador que el producto no fue encontrado.

4. Repartidores

- **RF4.1:** Los repartidores deben escuchar pedidos preparados desde una cola de mensajes específica una vez que los robots terminan su trabajo.
- **RF4.2:** Los repartidores deben simular hasta tres intentos de entrega por pedido, cada uno con una espera aleatoria de 10 a 20 segundos y una probabilidad de éxito configurable (p_entrega).
- **RF4.3:** Los repartidores deben informar al controlador del estado de cada intento de entrega (éxito o fallo).
- **RF4.4:** Si todos los intentos fallan, el repartidor debe notificar al controlador que la entrega no fue posible.

5. Pedidos

- **RF5.1:** Cada pedido debe incluir al menos: el ID del cliente, los IDs de los productos y el estado actual del pedido.
- **RF5.2:** El sistema debe definir los posibles estados de un pedido (ejemplo: creado, en almacén, en cinta, en reparto, entregado, cancelado) y sus transiciones.
- **RF5.3:** La cancelación de un pedido solo debe ser posible antes de que sea asignado a un repartidor, y los productos cancelados deben "volver" al almacén (simulado).

6. Colas de Mensajes

- **RF6.1:** El sistema debe usar RabbitMQ para la comunicación entre actores mediante colas de mensajes.
- **RF6.2:** Las colas deben tener nombres con prefijo GRUPO-PAREJA_ (ejemplo: 2321-01_robots) para evitar conflictos en el servidor compartido.
- **RF6.3:** El sistema debe declarar colas con durable=False y auto_delete=True durante el desarrollo para facilitar pruebas.

7. Mensajes

- **RF7.1:** El sistema debe definir una sintaxis clara para los mensajes entre actores (ejemplo: REGISTER client_id, MOVE order_id).
- **RF7.2:** Los mensajes deben incluir respuestas adecuadas (éxito o fallo), como REGISTERED client o MOVED order_id.

3.2 Requisitos No Funcionales

1. Tecnologías y Herramientas

- **RNF1.1:** El sistema debe implementarse utilizando RabbitMQ como sistema de colas de mensajes.
- **RNF1.2:** El sistema debe desarrollarse en Python.
- **RNF1.3:** Se recomienda usar la librería pika para RabbitMQ, uuid para generar IDs y argparse para la interfaz CLI.
- **RNF1.4:** Para pruebas locales, se puede usar un servidor RabbitMQ con la imagen Docker oficial; para desarrollo y entrega, debe usarse el servidor redes2.ii.uam.es.

2. Simulación

- **RNF2.1:** No es necesario modelar un registro de productos válidos; la disponibilidad de productos se simula con probabilidades.
- **RNF2.2:** Las operaciones de robots y repartidores deben simularse con esperas aleatorias y probabilidades configurables (p_almacen y p_entrega).

3. Persistencia

- **RNF3.1:** El estado del sistema (pedidos y clientes) debe persistirse usando ficheros (Pickle o CSV) o una base de datos, y los datos deben guardarse antes de finalizar la ejecución.

4. Diseño y Documentación

- **RNF4.1:** El sistema debe incluir un diagrama de clases que represente los actores y sus relaciones.
- **RNF4.2:** El sistema debe incluir un diagrama de estados que detalle las transiciones de los pedidos.
- **RNF4.3:** Se deben documentar tres casos de uso desde la perspectiva del controlador: pedido completado, pedido fallido por el robot y pedido cancelado.
- **RNF4.4:** Se debe entregar un documento tipo RFC con la sintaxis y formato de los mensajes.

5. Calidad del Código

- **RNF5.1:** El código debe organizarse en scripts ejecutables: launch_controller.py, launch_client.py, commandline_client.py, launch_robot.py, launch_delivery.py.
- **RNF5.2:** Se deben incluir scripts Bash para simular situaciones reales ejecutando los scripts Python en procesos separados.
- **RNF5.3:** Es recomendable implementar tests unitarios para cada actor.
- **RNF5.4:** Opcionalmente, se pueden incluir scripts Python que simulan actores en hilos (aunque no es obligatorio).

6. Usabilidad

- **RNF6.1:** La interfaz del cliente debe ser una CLI (no gráfica) para simular acciones desde la terminal.

7. Desarrollo y Pruebas

- **RNF7.1:** El sistema debe diseñarse para permitir pruebas aisladas de los actores usando colas de mensajes.
- **RNF7.2:** Las colas deben configurarse con durable=False y auto_delete=True para evitar conflictos durante el desarrollo.

8. Entregables

- **RNF8.1:** El sistema debe entregarse con código funcional y los documentos requeridos (diagramas, casos de uso, RFC, documento de requisitos).

4. IMPLEMENTACIÓN DE LA SOLUCIÓN

4.1 Arquitectura del Sistema El sistema usa una arquitectura distribuida basada en mensajería:

- Cliente: Interfaz de consola.
- Controlador: Coordina solicitudes y datos.
- Robot: Busca productos.
- Repartidor: Entrega pedidos.
- RabbitMQ: Comunicación asíncrona.

Diagrama de Arquitectura (descripción para PDF): Un diagrama de flujo muestra al Cliente conectado al Controlador vía `cola_rpc` (ida) y `cola_respuestas` (vuelta). El Controlador envía tareas a Robots (`robot_tasks`) y recibe respuestas (`robot_responses`), y a Repartidores (`delivery_tasks`) con respuestas (`delivery_responses`). El Controlador accede a archivos JSON (clientes, pedidos) y logs.

4.2 Estructura de Mensajes

El sistema implementado en el código utiliza RabbitMQ para la comunicación entre componentes (clientes, robots y repartidores). A continuación, se describe la sintaxis y formato de los mensajes intercambiados en las colas definidas, organizados por tipo de mensaje y su propósito:

1. Mensajes en la clase Cliente (Solicitudes de clientes)

Propósito: Los clientes envían solicitudes al controlador para realizar acciones como registro, login, hacer pedidos, ver pedidos o cancelar pedidos.

Formato general:

- **Cuerpo:** Un objeto JSON codificado como cadena de bytes (UTF-8).
- **Estructura JSON:**

```
{
  "accion": "<acción>",
  "<clave>": "<valor>",
  ...
}
```

- **Propiedades del mensaje:**

- reply_to: Cola donde el controlador enviará la respuesta.
- correlation_id: Identificador único para correlacionar solicitud y respuesta.

Acciones soportadas:

1. **Registrar** (accion: "registrar")

- **Formato:**

```
{
  "accion": "registrar",
  "cliente_username": "<nombre_usuario>",
  "cliente_password": "<contraseña_hasheada>"
}
```

- **Respuesta:**

```
{
  "exito": <true|false>,
  "mensaje": "<mensaje_descriptivo>",
  "cliente_id": "<id_cliente>" // Solo si exito es true
}
```

2. **Login** (accion: "login")

- **Formato:**

```
{
  "accion": "login",
  "cliente_username": "<nombre_usuario>",
  "cliente_password": "<contraseña_hasheada>"
}
```

- **Respuesta:**

```
{
  "exito": <true|false>,
  "mensaje": "<mensaje_descriptivo>",
  "cliente_id": "<id_cliente>", // Solo si exito es true
  "cliente_username": "<nombre_usuario>" // Solo si exito es true
}
```

3. **Hacer Pedido** (accion: "hacer_pedido")

- **Formato:**

```
{
  "accion": "hacer_pedido",
  "cliente_id": "<id_cliente>",
  "productos": ["<producto1>", "<producto2>", ...]
}
```

- **Respuesta:**

```
{
  "exito": <true|false>,
  "mensaje": "<mensaje_descriptivo>",
  "pedido_id": "<id_pedido>" // Solo si exito es true
}
```

4. **Ver Pedido** (accion: "ver_pedidos")

- **Formato:**

```
{
  "accion": "ver_pedidos",
  "cliente_id": "<id_cliente>"
}
```

- **Respuesta:**

```
{
  "exito": <true|false>,
  "mensaje": "<mensaje_descriptivo>", // Solo si exito es false
  "pedidos": [
    {
      "id": "<id_pedido>",
      "productos": ["<producto1>", "<producto2>", ...],
      "fecha": "<fecha>",
      "estado": "<estado>"
    },
    ...
  ]
}
```

5. **Cancelar Pedido** (accion: "cancelar_pedido")

- **Formato:**

```
{
  "accion": "cancelar_pedido",
  "cliente_id": "<id_cliente>",
  "pedido_id": "<id_pedido>"
}
```

○ **Respuesta:**

```
{
  "exito": <true|false>,
  "mensaje": "<mensaje_descriptivo>"
}
```

detalles adicionales:

- Los mensajes se envían mediante el método `enviar_solicitud`, que utiliza `basic_publish` para publicar en `cola_rpc`.
- El cliente espera una respuesta en su cola exclusiva (`cola_respuestas`) durante un tiempo máximo definido por `TIMEOUT` (valor en `config.py`). Si no llega la respuesta, se retorna `None`.
- La cola exclusiva se declara con `exclusive=True`, asegurando que solo el cliente que la creó pueda consumir sus mensajes.

Ejemplo de flujo:

- Cliente envía: `{"accion": "login", "cliente_username": "juan", "cliente_password": "pass123"}` con `correlation_id=uuid123` y `reply_to=cola_exclusiva_456`.
- Recibe: `{"exito": true, "mensaje": "Autenticación exitosa", "cliente_id": "uuid456", "cliente_username": "juan"}`.

2. Mensajes en la clase Repartidor (Tareas para repartidores)

La clase `Repartidor` consume mensajes de la cola `delivery_tasks` y envía respuestas a la cola `delivery_responses`. Estos mensajes ya fueron descritos en la sección del Controlador, pero aquí se detallan desde la perspectiva del repartidor.

Colas utilizadas:

- `delivery_tasks`: Recibe tareas de entrega.
- `delivery_responses`: Envía respuestas sobre el estado de la entrega.

Mensajes recibidos (`delivery_tasks`)

- **Formato:**

```
DISTRIBUTE <pedido_id>
```

- **Propósito:** Indica al repartidor que debe iniciar la entrega del pedido identificado por <pedido_id>.
- **Ejemplo:**
 - DISTRIBUTE pedido123: Instruye al repartidor a distribuir el pedido pedido123.

Mensajes enviados (delivery_responses)

- **Formato general:**

```
<accion> <pedido_id> [detalles_adicionales]
```

- **Acciones posibles:**
 - DISTRIBUTING:**
 - **Formato:** DISTRIBUTING <pedido_id>
 - **Propósito:** Indica que el repartidor ha iniciado la entrega.
 - **Ejemplo:** DISTRIBUTING pedido123
 - DISTRIBUTED:**
 - **Formato:** DISTRIBUTED <pedido_id>
 - **Propósito:** Confirma que el pedido fue entregado exitosamente.
 - **Ejemplo:** DISTRIBUTED pedido123
 - DELIVERY_FAILED:**
 - **Formato:** DELIVERY_FAILED <pedido_id> Intento <número>
 - **Propósito:** Reporta un intento de entrega fallido, incluyendo el número de intento.
 - **Ejemplo:** DELIVERY_FAILED pedido123 Intento 1
 - DELIVERY_FAILED_FINAL:**
 - **Formato:** DELIVERY_FAILED_FINAL <pedido_id>
 - **Propósito:** Indica que la entrega falló tras tres intentos, resultando en la cancelación del pedido.
 - **Ejemplo:** DELIVERY_FAILED_FINAL pedido123

Detalles adicionales:

- Los mensajes se envían mediante basic_publish sin propiedades específicas (sin reply_to ni correlation_id).
- El repartidor simula el tiempo de entrega con una espera aleatoria entre 10 y 20 segundos.
- La probabilidad de éxito de la entrega está definida por P_ENTREGA (en config.py). Si falla, se realizan hasta tres intentos antes de enviar DELIVERY_FAILED_FINAL.
- Los mensajes recibidos se confirman con basic_ack si se procesan correctamente o basic_nack si hay un error o el mensaje es inesperado.

Ejemplo de flujo:

- **Recibe:** DISTRIBUTE pedido123.
- **Envía:** DISTRIBUTING pedido123.
- **Tras un intento fallido:** DELIVERY_FAILED pedido123 Intento 1.
- **Si todos los intentos fallan:** DELIVERY_FAILED_FINAL pedido123.
- **Si la entrega es exitosa:** DISTRIBUTED pedido123.

3. Mensajes en la clase Robot (Tareas para repartidores)

La clase Robot consume mensajes de la cola robot_tasks y envía respuestas a la cola robot_responses. Estos mensajes también fueron descritos en la sección del Controlador, pero se detallan aquí desde la perspectiva del robot.

Colas utilizadas:

- **robot_tasks:** Recibe tareas para buscar productos.
- **robot_responses:** Envía resultados de la búsqueda.

Mensajes recibidos (robot_tasks)

- **Formato:**

```
Move <pedido_id> <producto>
```

- **Propósito:** Instruye al robot a buscar un producto específico para un pedido.
- **Ejemplo:**
 - Move pedido123 productoA: Indica al robot que busque el producto productoA para el pedido pedido123.

Mensajes enviados (robot_responses)

- **Formato:**

```
<accion> <pedido_id> <producto>
```

- **Acciones posibles:**
 1. **MOVED:**
 - **Formato:** MOVED <pedido_id> <producto>
 - **Propósito:** Indica que el producto fue encontrado y movido.
 - **Ejemplo:** MOVED pedido123 productoA
 2. **NOT_FOUND:**
 - **Formato:** NOT_FOUND <pedido_id> <producto>
 - **Propósito:** Indica que el producto no fue encontrado.
 - **Ejemplo:** NOT_FOUND pedido123 productoB

Detalles adicionales:

- Los mensajes se envían mediante `basic_publish` sin propiedades específicas.
- El robot simula el tiempo de búsqueda con una espera aleatoria entre 5 y 10 segundos.
- La probabilidad de encontrar el producto está definida por `P_ALMACEN` (en `config.py`).
- Los mensajes recibidos se confirman con `basic_ack` si se procesan correctamente o `basic_nack` si hay un error o el mensaje es inesperado.

Ejemplo de flujo:

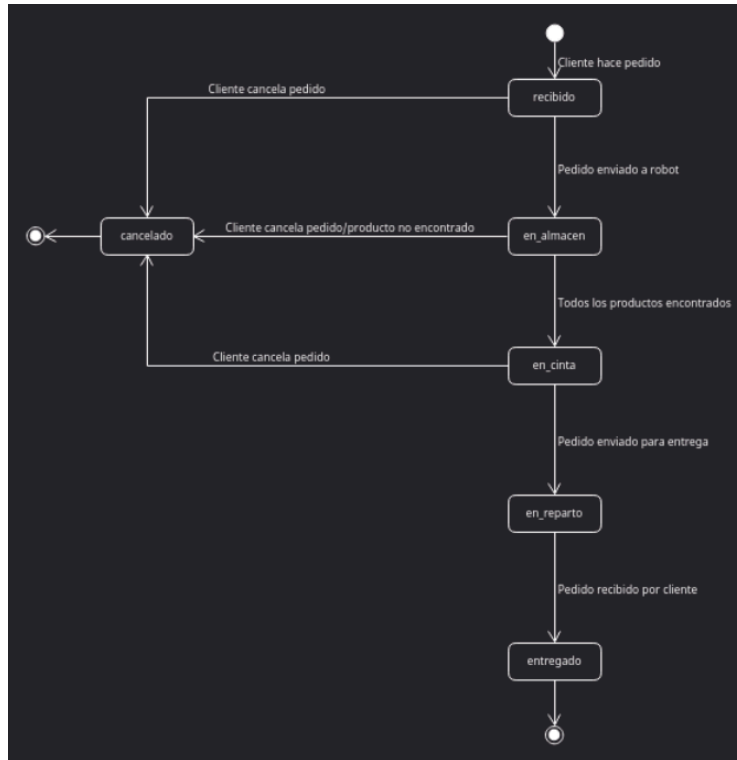
- **Recibe: `Move pedido123 productoA`.**
- **Si encuentra el producto: Envía `MOVED pedido123 productoA`.**
- **Si no lo encuentra: Envía `NOT_FOUND pedido123 productoA`.**

4.3 Implementación Técnica

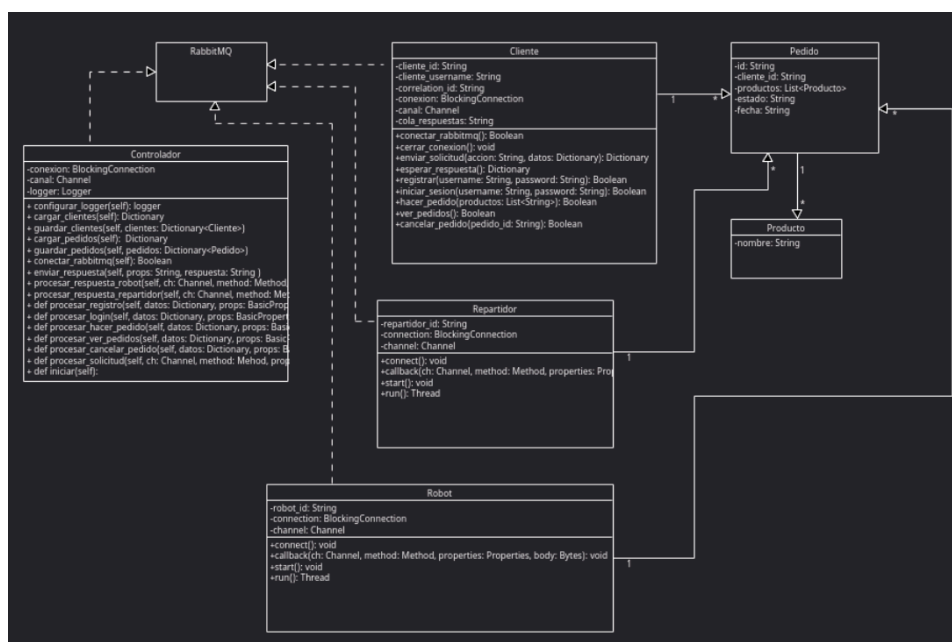
- Lenguaje: Python 3.6+.
- Bibliotecas: `pika` (RabbitMQ), `uuid`, `json`, `logging`.
- Estructura:
 - Controlador: Gestiona colas, datos, lógica.
 - Cliente: Interfaz y solicitudes RPC.
 - Robot: Búsqueda con `P_ALMACEN`.
 - Repartidor: Entrega con `P_ENTREGA`.
 - `config.py`: Parámetros de configuración.
- Almacenamiento: JSON (`ARCHIVO_CLIENTES`, `ARCHIVO_PEDIDOS`), logs (`logs/controlador.log`).

5. DIAGRAMAS

5.1 Diagrama de Estados



5.2 Diagrama de Clases



5.3 Casos de Uso

Caso de Uso 1: Pedido Completado Hasta el Final

Nombre: Procesamiento exitoso de pedido completo

Actores: Cliente, Controlador, Robot

Descripción: Este caso de uso describe el proceso exitoso de un pedido desde que se crea hasta que se entrega al cliente.

Precondiciones:

El cliente está registrado y autenticado en el sistema

Los productos solicitados están disponibles en el almacén

Flujo Principal:

- El cliente solicita hacer un pedido con una lista de productos
- El controlador registra el nuevo pedido con estado "recibido"
- El controlador actualiza el estado del pedido a "en_almacen"
- El controlador envía solicitudes de búsqueda para cada producto a los robots
- Los robots encuentran todos los productos solicitados
- El controlador verifica que todos los productos han sido encontrados
- El controlador actualiza el estado del pedido a "en_cinta"
- El controlador envía solicitud a los repartidores
- Un repartidor es encontrado
- El controlador actualiza el estado del pedido a "en_reparto"
- El pedido es entregado al cliente
- El controlador actualiza el estado del pedido a "entregado"

Postcondiciones:

- El pedido queda registrado como "entregado"
- El cliente recibe todos los productos solicitados

Flujos Alternativos: Ninguno en este caso de uso (los escenarios de error se tratan en otros casos de uso)

Caso de Uso 2: Robot No Encuentra un Producto

Nombre: Cancelación automática por producto no disponible

Actores: Cliente, Controlador, Robot

Descripción: Este caso de uso describe el proceso cuando uno o más productos de un pedido no están disponibles en el almacén.

Precondiciones:

- El cliente está registrado y autenticado en el sistema
- El cliente ha solicitado al menos un producto que no está disponible en el almacén

Flujo Principal:

- El cliente solicita hacer un pedido con una lista de productos
- El controlador registra el nuevo pedido con estado "recibido"
- El controlador actualiza el estado del pedido a "en_almacen"
- El controlador envía solicitudes de búsqueda para cada producto a los robots
- Para uno o más productos, el robot no encuentra el producto en el almacén
- El robot envía una notificación "NOT_FOUND" al controlador
- El controlador marca el producto como "no_encontrado" en el registro del pedido
- El controlador identifica que existen productos no disponibles
- El controlador cambia automáticamente el estado del pedido a "cancelado"
- El controlador registra el motivo de la cancelación

Postcondiciones:

- El pedido queda registrado como "cancelado"
- Se registra el motivo específico de la cancelación (producto no encontrado)
- No se procesan más productos para este pedido

Flujos Alternativos: Ninguno en este caso de uso

Caso de Uso 3: Cliente Cancela Pedido Antes del Reparto

Nombre: Cancelación manual de pedido por el cliente

Actores: Cliente, Controlador

Descripción: Este caso de uso describe el proceso cuando un cliente decide cancelar un pedido que ya está en procesamiento pero aún no ha salido a reparto.

Precondiciones:

- El cliente está registrado y autenticado en el sistema
- El cliente tiene un pedido activo en estado cancelable ("recibido", "en_almacen" o "en_cinta")
- Todos los productos del pedido han sido encontrados en el almacén

Flujo Principal:

- El cliente solicita ver sus pedidos
- El sistema muestra la lista de pedidos del cliente con sus estados actuales
- El cliente selecciona un pedido específico para cancelar
- El cliente envía la solicitud de cancelación del pedido
- El controlador verifica que el pedido pertenece al cliente
- El controlador verifica que el pedido está en un estado cancelable
- El controlador cambia el estado del pedido a "cancelado"
- El controlador registra la cancelación en el historial del pedido
- El sistema notifica al cliente que la cancelación fue exitosa

Postcondiciones:

- El pedido queda registrado como "cancelado"
- Los productos vuelven a estar disponibles en el almacén
- El cliente recibe confirmación de la cancelación exitosa

Flujos Alternativos:

El pedido está en un estado no cancelable

- En el paso 6, si el controlador detecta que el pedido está en estado "en_reparto" o "entregado"
- El sistema notifica al cliente que el pedido ya no puede ser cancelado
- El caso de uso termina sin cancelar el pedido

El pedido ya está cancelado

- En el paso 6, si el controlador detecta que el pedido ya está en estado "cancelado", el sistema notifica al cliente que el pedido ya fue cancelado previamente
- El caso de uso termina sin cambios adicionales

6. PLAN DE PRUEBAS

El plan de pruebas valida el cumplimiento de los requisitos funcionales (RF1.1 a RF7.2) y no funcionales (RNF1.1 a RNF8.1) del sistema Saimazoom. Las pruebas se ejecutan mediante scripts automatizados que simulan diversos escenarios, desde operaciones básicas hasta condiciones de alta carga y fallos, utilizando RabbitMQ como sistema de mensajería. El script `check_orders.py` verifica que todos los pedidos de un usuario estén en estado "cancelado" o "entregado" para considerar una prueba exitosa. Para garantizar un entorno lógico y sin errores, las pruebas deben ejecutarse en orden, comenzando con el registro del usuario "daniel" (contraseña "daniel1234") mediante `run_register_login.sh`.

6.1 Pruebas Funcionales

Las pruebas funcionales verifican que cada requisito funcional se cumpla correctamente, cubriendo las interacciones entre el controlador, clientes, robots, repartidores, pedidos, colas de mensajes, y formatos de mensajes.

- **RF1.1: Gestión de pedidos y coordinación vía RabbitMQ**
 - **Descripción:** Verificar que el controlador coordina la comunicación con clientes, robots, y repartidores a través de colas RabbitMQ.
 - **Prueba:** Ejecutar `run_basics.sh`, que inicia `launch_controler.py`, `launch_robot.py`, `launch_repartidor.py`, y `proof_delivery.py`. El controlador procesa solicitudes de clientes (`cola_rpc`), envía tareas a robots (`robot_tasks`) y repartidores (`delivery_tasks`), y recibe respuestas (`robot_responses`, `delivery_responses`).

- **Criterio de Éxito:** Los logs del controlador muestran mensajes enviados y recibidos correctamente en las colas. `check_orders.py` confirma que los pedidos alcanzan estados "entregado" o "cancelado".
 - **Script:** `run_basics.sh`
- **RF1.2: Registro de pedidos con estado**
 - **Descripción:** Verificar que el controlador registra todos los pedidos con su estado actual.
 - **Prueba:** Ejecutar `run_basics.sh`, donde `proof_delivery.py` crea un pedido. Inspeccionar `ARCHIVO_PEDIDOS` para verificar los estados (e.g., "recibido", "en_almacen", "en_cinta", "en_reparto", "entregado").
 - **Criterio de Éxito:** Cada pedido en `ARCHIVO_PEDIDOS` incluye ID, cliente, productos, y estado actual, con transiciones correctas (ver RF5.2).
 - **Script:** `run_basics.sh`
- **RF1.3: Registro de clientes y asociación de pedidos**
 - **Descripción:** Verificar que el controlador registra clientes y asocia pedidos a ellos.
 - **Prueba:** Ejecutar `run_register_login.sh` para registrar al usuario "daniel", seguido de `run_basics.sh` para crear un pedido. Inspeccionar `ARCHIVO_CLIENTES` y `ARCHIVO_PEDIDOS` para confirmar la asociación.
 - **Criterio de Éxito:** `ARCHIVO_CLIENTES` contiene el cliente "daniel" con un ID único. `ARCHIVO_PEDIDOS` asocia cada pedido al `cliente_id` correcto.
 - **Scripts:** `run_register_login.sh`, `run_basics.sh`
- **RF1.4: Rechazo de pedidos de clientes no registrados**
 - **Descripción:** Verificar que el controlador rechaza pedidos de clientes no registrados.
 - **Prueba:** Modificar `proof_delivery.py` para enviar un pedido con un `cliente_id` no registrado. Ejecutar `run_basics.sh` y verificar la respuesta.
 - **Criterio de Éxito:** El controlador responde con `{"exito": false, "mensaje": "Cliente no registrado"}`. El pedido no se registra en `ARCHIVO_PEDIDOS`.
 - **Script:** `run_basics.sh` (con modificaciones)
- **RF1.5: Persistencia de estado (pedidos y clientes)**
 - **Descripción:** Verificar que el controlador persiste el estado en archivos JSON y lo guarda antes de finalizar.
 - **Prueba:** Ejecutar `run_basics.sh` y `run_multiple_clients.sh`. Detener el controlador (`launch_controler.py`) y verificar que `ARCHIVO_CLIENTES` y `ARCHIVO_PEDIDOS` reflejan el estado final.
 - **Criterio de Éxito:** Los archivos JSON contienen datos actualizados de clientes y pedidos tras la ejecución. Los datos persisten entre ejecuciones.
 - **Scripts:** `run_basics.sh`, `run_multiple_clients.sh`
- **RF1.6: Recepción de confirmaciones/errores de robots y repartidores**
 - **Descripción:** Verificar que el controlador actualiza los estados de los pedidos según las respuestas de robots y repartidores.
 - **Prueba:** Ejecutar `run_basics.sh` y `run_failure.sh`. En `run_basics.sh`, los robots envían `MOVED/NOT_FOUND` y los repartidores envían `DISTRIBUTED/DELIVERY_FAILED`. En `run_failure.sh`, los pedidos se cancelan por falta de repartidores.

- **Criterio de Éxito:** Los logs del controlador muestran respuestas procesadas (MOVED, NOT_FOUND, DISTRIBUTED, DELIVERY_FAILED_FINAL). Los pedidos reflejan estados correctos en ARCHIVO_PEDIDOS.
- **Scripts:** run_basics.sh, run_failure.sh
- **RF2.1: Registro de clientes**
 - **Descripción:** Verificar que los clientes pueden registrarse enviando un mensaje al controlador.
 - **Prueba:** Ejecutar run_register_login.sh, que inicia register_login.py para registrar al usuario "daniel".
 - **Criterio de Éxito:** Respuesta {"exito": true, "cliente_id": "<uuid>"}. ARCHIVO_CLIENTES incluye al cliente registrado.
 - **Script:** run_register_login.sh
- **RF2.2: Creación, consulta, y cancelación de pedidos**
 - **Descripción:** Verificar que los clientes registrados pueden crear pedidos, consultar su estado, y cancelarlos si no están asignados a repartidores.
 - **Prueba:**
 - **Creación y consulta:** Ejecutar run_basics.sh, donde proof_delivery.py crea un pedido y check_orders.py consulta su estado.
 - **Cancelación:** Modificar proof_delivery.py para cancelar un pedido en estado "en_almacen" y ejecutar run_failure.sh (sin repartidores) para probar la cancelación.
 - **Criterio de Éxito:**
 - Creación: Respuesta {"exito": true, "pedido_id": "<uuid>"}
 - Consulta: check_orders.py muestra pedidos con estados "cancelado" o "entregado".
 - Cancelación: Respuesta {"exito": true, "mensaje": "Pedido cancelado exitosamente"} para estados permitidos; {"exito": false} para estados no permitidos.
 - **Scripts:** run_basics.sh, run_failure.sh (con modificaciones)
- **RF2.3: Interfaz CLI para clientes**
 - **Descripción:** Verificar que los clientes interactúan mediante una interfaz de línea de comandos.
 - **Prueba:** Ejecutar run_basics.sh y run_multiple_clients.sh, que usan proof_delivery.py (o un script CLI similar) para simular acciones de clientes desde la terminal.
 - **Criterio de Éxito:** La CLI permite registrar, autenticar, crear pedidos, y consultar estados sin errores. Los logs del cliente muestran interacciones correctas.
 - **Scripts:** run_basics.sh, run_multiple_clients.sh
- **RF3.1: Robots escuchan peticiones en cola específica**
 - **Descripción:** Verificar que los robots consumen tareas de la cola GRUPO-PAREJA_robot_tasks.
 - **Prueba:** Ejecutar run_basics.sh y run_high_load.sh, que inician launch_robot.py. El controlador envía tareas a robot_tasks, y los robots las procesan.

- **Criterio de Éxito:** Los logs del controlador muestran tareas enviadas a GRUPO-PAREJA_robot_tasks y respuestas recibidas en GRUPO-PAREJA_robot_responses.
 - **Scripts:** run_basics.sh, run_high_load.sh
- **RF3.2: Simulación de búsqueda con espera aleatoria**
 - **Descripción:** Verificar que los robots simulan la búsqueda con esperas de 5 a 10 segundos.
 - **Prueba:** Ejecutar run_basics.sh. Inspeccionar los logs de launch_robot.py para confirmar los tiempos de espera.
 - **Criterio de Éxito:** Los logs muestran esperas aleatorias entre 5 y 10 segundos por tarea de búsqueda.
 - **Script:** run_basics.sh
- **RF3.3: Simulación de éxito/fallo con probabilidad p_almacen**
 - **Descripción:** Verificar que los robots simulan éxito (MOVED) o fallo (NOT_FOUND) según p_almacen.
 - **Prueba:** Ejecutar run_failure.sh, que aumenta la probabilidad de fallos (NOT_FOUND) debido a la configuración de p_almacen.
 - **Criterio de Éxito:** Los logs del controlador muestran una mezcla de MOVED y NOT_FOUND, coherente con p_almacen. Los pedidos con productos no encontrados se cancelan.
 - **Script:** run_failure.sh
- **RF3.4: Notificación de producto movido**
 - **Descripción:** Verificar que los robots notifican al controlador cuando un producto es movido.
 - **Prueba:** Ejecutar run_basics.sh. Los robots envían MOVED <pedido_id> <producto> a robot_responses.
 - **Criterio de Éxito:** Los logs del controlador muestran MOVED para cada producto encontrado, y el pedido avanza a "en_cinta".
 - **Script:** run_basics.sh
- **RF3.5: Notificación de producto no encontrado**
 - **Descripción:** Verificar que los robots notifican al controlador cuando un producto no se encuentra.
 - **Prueba:** Ejecutar run_failure.sh. Los robots envían NOT_FOUND <pedido_id> <producto> debido a p_almacen.
 - **Criterio de Éxito:** Los logs del controlador muestran NOT_FOUND, y el pedido se cancela en ARCHIVO_PEDIDOS.
 - **Script:** run_failure.sh
- **RF4.1: Repartidores escuchan pedidos en cola específica**
 - **Descripción:** Verificar que los repartidores consumen tareas de la cola GRUPO-PAREJA_delivery_tasks.
 - **Prueba:** Ejecutar run_basics.sh y run_high_load.sh, que inician launch_repartidor.py. El controlador envía tareas a delivery_tasks.
 - **Criterio de Éxito:** Los logs del controlador muestran tareas enviadas a GRUPO-PAREJA_delivery_tasks y respuestas en GRUPO-PAREJA_delivery_responses.
 - **Scripts:** run_basics.sh, run_high_load.sh
- **RF4.2: Simulación de hasta tres intentos de entrega**

- **Descripción:** Verificar que los repartidores simulan hasta tres intentos de entrega con esperas de 10 a 20 segundos y probabilidad $p_{entrega}$.
 - **Prueba:** Ejecutar `run_basics.sh`. Inspeccionar los logs de `launch_repartidor.py` para confirmar los intentos y tiempos.
 - **Criterio de Éxito:** Los logs muestran `DELIVERY_FAILED <pedido_id>` Intento X (hasta tres veces) o `DISTRIBUTED <pedido_id>`, con esperas entre 10 y 20 segundos.
 - **Script:** `run_basics.sh`
- **RF4.3: Informe de estado de intentos de entrega**
 - **Descripción:** Verificar que los repartidores informan el estado de cada intento al controlador.
 - **Prueba:** Ejecutar `run_basics.sh` y `run_high_load.sh`. Los repartidores envían `DISTRIBUTING`, `DISTRIBUTED`, o `DELIVERY_FAILED` a `delivery_responses`.
 - **Criterio de Éxito:** Los logs del controlador muestran todas las respuestas (`DISTRIBUTING`, `DISTRIBUTED`, `DELIVERY_FAILED`).
 - **Scripts:** `run_basics.sh`, `run_high_load.sh`
- **RF4.4: Notificación de entrega fallida**
 - **Descripción:** Verificar que los repartidores notifican al controlador si la entrega falla tras tres intentos.
 - **Prueba:** Ejecutar `run_basics.sh` con una baja $p_{entrega}$ para forzar fallos. Los repartidores envían `DELIVERY_FAILED_FINAL <pedido_id>`.
 - **Criterio de Éxito:** Los logs del controlador muestran `DELIVERY_FAILED_FINAL`, y el pedido se cancela en `ARCHIVO_PEDIDOS`.
 - **Script:** `run_basics.sh` (con ajustes en $p_{entrega}$)
- **RF5.1: Contenido de pedidos**
 - **Descripción:** Verificar que cada pedido incluye ID del cliente, productos, y estado.
 - **Prueba:** Ejecutar `run_basics.sh`. Inspeccionar `ARCHIVO_PEDIDOS` tras crear un pedido con `proof_delivery.py`.
 - **Criterio de Éxito:** Cada pedido en `ARCHIVO_PEDIDOS` contiene `cliente_id`, lista de productos, y estado actual.
 - **Script:** `run_basics.sh`
- **RF5.2: Estados y transiciones de pedidos**
 - **Descripción:** Verificar los estados de los pedidos ("creado", "en_almacen", "en_cinta", "en_reparto", "entregado", "cancelado") y sus transiciones.
 - **Prueba:** Ejecutar `run_basics.sh` para transiciones completas (creado → entregado) y `run_failure.sh` para cancelaciones (creado → cancelado).
 - **Criterio de Éxito:** Los logs del controlador y `ARCHIVO_PEDIDOS` reflejan transiciones correctas según el diagrama de estados (RNF4.2).
 - **Scripts:** `run_basics.sh`, `run_failure.sh`
- **RF5.3: Cancelación de pedidos**
 - **Descripción:** Verificar que los pedidos solo se cancelan antes de ser asignados a repartidores, y los productos "vuelven" al almacén (simulado).
 - **Prueba:** Modificar `proof_delivery.py` para cancelar un pedido en estado "en_almacen" y ejecutar `run_failure.sh` (sin repartidores). Verificar que el controlador marca el pedido como "cancelado".

- **Criterio de Éxito:** Respuesta {"exito": true, "mensaje": "Pedido cancelado exitosamente"}. ARCHIVO_PEDIDOS muestra el estado "cancelado".
 - **Script:** run_failure.sh (con modificaciones)
- **RF6.1: Uso de RabbitMQ**
 - **Descripción:** Verificar que el sistema usa RabbitMQ para la comunicación entre actores.
 - **Prueba:** Ejecutar run_basics.sh, run_high_load.sh, y run_multiple_clients.sh. Usar rabbitmqctl para inspeccionar las colas.
 - **Criterio de Éxito:** Las colas GRUPO-PAREJA_cola_rpc, GRUPO-PAREJA_robot_tasks, GRUPO-PAREJA_delivery_tasks, etc., procesan mensajes correctamente.
 - **Scripts:** run_basics.sh, run_high_load.sh, run_multiple_clients.sh
- **RF6.2: Nombres de colas con prefijo GRUPO-PAREJA_**
 - **Descripción:** Verificar que las colas usan el prefijo GRUPO-PAREJA_ para evitar conflictos.
 - **Prueba:** Ejecutar run_basics.sh y usar rabbitmqctl list_queues para inspeccionar los nombres de las colas.
 - **Criterio de Éxito:** Todas las colas tienen nombres como GRUPO-PAREJA_cola_rpc, GRUPO-PAREJA_robot_tasks, etc.
 - **Script:** run_basics.sh
- **RF6.3: Colas con durable=False y auto_delete=True**
 - **Descripción:** Verificar que las colas se declaran con durable=False y auto_delete=True durante el desarrollo.
 - **Prueba:** Ejecutar run_basics.sh. Inspeccionar el código de launch_controler.py, launch_robot.py, y launch_repartidor.py para confirmar la configuración de las colas.
 - **Criterio de Éxito:** Las colas se eliminan automáticamente al desconectar los consumidores, y no persisten tras reiniciar RabbitMQ.
 - **Script:** run_basics.sh
- **RF7.1: Sintaxis clara de mensajes**
 - **Descripción:** Verificar que los mensajes tienen una sintaxis definida (e.g., MOVE <pedido_id> <producto>, REGISTER <cliente_id>).
 - **Prueba:** Ejecutar run_basics.sh y run_register_login.sh. Inspeccionar los logs del controlador para confirmar los formatos de mensajes.
 - **Criterio de Éxito:** Los logs muestran mensajes con la sintaxis especificada en el documento RFC (RNF4.4), como MOVE pedido123 Manzanas, REGISTERED daniel.
 - **Scripts:** run_basics.sh, run_register_login.sh
- **RF7.2: Respuestas adecuadas de mensajes**
 - **Descripción:** Verificar que los mensajes incluyen respuestas de éxito o fallo (e.g., REGISTERED, MOVED, NOT_FOUND).
 - **Prueba:** Ejecutar run_basics.sh y run_failure.sh. Verificar las respuestas en los logs del controlador y del cliente.
 - **Criterio de Éxito:** Respuestas como {"exito": true, "mensaje": "Usuario registrado"}, MOVED pedido123 Manzanas, o NOT_FOUND pedido123 Peras se procesan correctamente.
 - **Scripts:** run_basics.sh, run_failure.sh

6.2 Pruebas No Funcionales

Las pruebas no funcionales evalúan el comportamiento del sistema en términos de tecnologías, simulación, persistencia, diseño, calidad del código, usabilidad, y desarrollo/pruebas.

- **RNF1.1–RNF1.4: Tecnologías (RabbitMQ, Python, pika, servidor)**
 - **Descripción:** Verificar que el sistema usa RabbitMQ, Python, la librería pika, y el servidor redes2.ii.uam.es (o Docker localmente).
 - **Prueba:** Ejecutar run_basics.sh en el servidor redes2.ii.uam.es o en un contenedor Docker local. Inspeccionar el código para confirmar el uso de pika y Python.
 - **Criterio de Éxito:** El sistema funciona sin errores en el servidor especificado. El código usa pika para RabbitMQ y uuid para IDs.
 - **Script:** run_basics.sh
- **RNF2.1–RNF2.2: Simulación de disponibilidad y operaciones**
 - **Descripción:** Verificar que la disponibilidad de productos y las operaciones de robots/repartidores se simulan con probabilidades y esperas aleatorias.
 - **Prueba:** Ejecutar run_basics.sh y run_failure.sh. Configurar p_almacen y p_entrega en config.py para probar diferentes probabilidades.
 - **Criterio de Éxito:** Los logs muestran esperas aleatorias (5-10 segundos para robots, 10-20 segundos para repartidores) y resultados coherentes con p_almacen/p_entrega.
 - **Scripts:** run_basics.sh, run_failure.sh
- **RNF3.1: Persistencia en archivos**
 - **Descripción:** Verificar que el estado se persiste en archivos JSON (ARCHIVO_CLIENTES, ARCHIVO_PEDIDOS).
 - **Prueba:** Ejecutar run_basics.sh y run_multiple_clients.sh. Detener el controlador y verificar los archivos JSON.
 - **Criterio de Éxito:** Los archivos contienen datos actualizados de clientes y pedidos, persistentes entre ejecuciones.
 - **Scripts:** run_basics.sh, run_multiple_clients.sh
- **RNF4.1–RNF4.4: Diseño y documentación**
 - **Descripción:** Verificar la existencia de diagramas de clases, estados, casos de uso, y documento RFC.
 - **Prueba:** Revisar los entregables documentales (diagramas en la sección 5, casos de uso en 2.5, RFC en 4.2). Ejecutar run_basics.sh, run_failure.sh, y run_multiple_clients.sh para validar los casos de uso (pedido completado, fallido, cancelado).
 - **Criterio de Éxito:** Los diagramas y casos de uso están completos. Las pruebas reflejan los escenarios documentados (e.g., cancelación en run_failure.sh).
 - **Scripts:** run_basics.sh, run_failure.sh, run_multiple_clients.sh
- **RNF5.1–RNF5.4: Calidad del código**

- **Descripción:** Verificar que el código está organizado en scripts ejecutables, incluye scripts Bash, y opcionalmente tests unitarios.
- **Prueba:** Inspeccionar la estructura del código (launch_controler.py, launch_client.py, launch_robot.py, launch_repartidor.py). Ejecutar run_basics.sh, run_high_load.sh, run_multiple_clients.sh, y run_failure.sh para validar los scripts Bash. Usar pytest para pruebas unitarias (si implementadas).
- **Criterio de Éxito:** Los scripts son modulares y ejecutables. Los scripts Bash orquestan procesos correctamente. Las pruebas unitarias (si existen) cubren métodos clave.
- **Scripts:** run_basics.sh, run_high_load.sh, run_multiple_clients.sh, run_failure.sh
- **RNF6.1: Usabilidad de la CLI**
 - **Descripción:** Verificar que la interfaz CLI es funcional para las acciones del cliente.
 - **Prueba:** Ejecutar run_basics.sh y run_multiple_clients.sh, que usan proof_delivery.py como CLI.
 - **Criterio de Éxito:** La CLI permite registrar, autenticar, crear pedidos, y consultar estados sin errores.
 - **Scripts:** run_basics.sh, run_multiple_clients.sh
- **RNF7.1–RNF7.2: Desarrollo y pruebas aisladas**
 - **Descripción:** Verificar que los actores pueden probarse de forma aislada y que las colas están configuradas correctamente.
 - **Prueba:** Ejecutar run_basics.sh para pruebas integradas y run_failure.sh para pruebas aisladas (sin repartidores). Inspeccionar la configuración de colas (durable=False, auto_delete=True).
 - **Criterio de Éxito:** Cada actor (controlador, robot, repartidor, cliente) funciona de forma aislada. Las colas se eliminan tras las pruebas.
 - **Scripts:** run_basics.sh, run_failure.sh
- **RNF8.1: Entregables**
 - **Descripción:** Verificar que el sistema incluye código funcional y documentación requerida.
 - **Prueba:** Revisar el repositorio entregado, incluyendo código, diagramas, casos de uso, y RFC. Ejecutar todos los scripts de prueba para confirmar funcionalidad.
 - **Criterio de Éxito:** El código es funcional, y los documentos están completos según RNF4.1–RNF4.4.
 - **Scripts:** run_register_login.sh, run_basics.sh, run_high_load.sh, run_multiple_clients.sh, run_failure.sh

6.3 Estrategia de Pruebas

- **Pruebas Unitarias:**
 - Métodos clave de las clases Controlador (e.g., procesar_registro, procesar_hacer_pedido), Cliente (e.g., registrar, cancelar_pedido), Robot (e.g., callback), y Repartidor (e.g., callback) se prueban con pytest.
 - Ejemplo: Verificar que procesar_registro rechaza nombres de usuario duplicados (RF1.4) y que callback del robot respeta p_almacen (RF3.3).

- **Pruebas de Integración:**
 - Los scripts `run_basics.sh`, `run_high_load.sh`, `run_multiple_clients.sh`, y `run_failure.sh` validan la comunicación entre actores a través de colas RabbitMQ, cubriendo el flujo completo desde el registro hasta la entrega o cancelación de pedidos.
 - `check_orders.py` asegura que los estados finales de los pedidos sean "cancelado" o "entregado".
- **Pruebas de Estrés:**
 - `run_high_load.sh` simula alta carga con múltiples clientes, robots, y repartidores, verificando la escalabilidad (RNF1.1, RNF7.1).
 - `run_multiple_clients.sh` evalúa el comportamiento con múltiples clientes concurrentes.
- **Pruebas de Fallo:**
 - `run_failure.sh` simula un escenario con cuellos de botella (sin repartidores), validando la cancelación de pedidos (RF5.3) y la robustez del sistema.
- **Herramientas:**
 - `pytest`: Para pruebas unitarias.
 - `rabbitmqctl`: Para monitorear colas y detectar conflictos.
 - `check_orders.py`: Para verificar estados de pedidos.
 - Scripts Bash (`run_*.sh`): Para orquestar pruebas y gestionar procesos.
- **Orden de Ejecución:**
 - **`run_register_login.sh`**: Registra y autentica al usuario "daniel" para un entorno inicial válido (RF2.1, RF1.3).
 - **`run_basics.sh`**: Prueba un escenario básico con un cliente, robot, y repartidor (RF1.1, RF2.2, RF3.1–RF3.5, RF4.1–RF4.4, RF5.1–RF5.2).
 - **`run_multiple_clients.sh`**: Evalúa múltiples clientes concurrentes (RF2.2, RF5.1).
 - **`run_high_load.sh`**: Simula alta carga con múltiples componentes (RNF1.1, RNF7.1).
 - **`run_failure.sh`**: Prueba fallos para verificar cancelaciones y robustez (RF5.3, RF3.5, RF1.6).
- **Preparación del Entorno:**
 - Verificar que RabbitMQ esté activo (`sudo rabbitmqctl status`) en `redes2.ii.uam.es` o un contenedor Docker (RNF1.4).
 - Configurar `config.py` con `RABBITMQ_HOST`, `p_almacen`, `p_entrega`, y prefijo `GRUPO-PAREJA_` (RF6.2).
 - Crear directorio `logs/` para almacenar registros (RNF5.1).
 - Instalar dependencias: `pip install pika` (RNF1.3).
- **Criterio de Éxito General:**
 - Todas las pruebas funcionales cumplen los criterios definidos para RF1.1–RF7.2.
 - Las pruebas no funcionales validan el uso de tecnologías (RNF1.1–RNF1.4), simulación (RNF2.1–RNF2.2), persistencia (RNF3.1), diseño (RNF4.1–RNF4.4), calidad del código (RNF5.1–RNF5.4), usabilidad (RNF6.1), y desarrollo/pruebas (RNF7.1–RNF7.2).
 - `check_orders.py` confirma que todos los pedidos están en estado "cancelado" o "entregado" al final de cada prueba.

7. SUPOSICIONES DE IMPLEMENTACIÓN

- Robots y repartidores simulados (integrar con hardware/APIs en producción).
- Autenticación sin encriptación avanzada.
- Tiempos de procesamiento simulados.

8. APÉNDICES

Apéndice A

- Pedido: Solicitud de productos.
- Estado del pedido: recibido, en_almacen, en_cinta, en_reparto, entregado, cancelado.
- Cola: Estructura de mensajería en RabbitMQ.